BUILD AN AGENT CREW



F.A.M-E.D. | Assignment3 Submission

Assignment 3 | Piero L. S. Folly | Khi-Rho Studios | 1 November 2026

F.A.M-E.D. (Fire Age Main-Eterra Dungeoneers) is a cooperative exploration and deck-building game. The submission contains the crew roles, orchestration evidence, card data, checks and an existing Bauglor combat prototype.

The report connects nine prototype results and my play feedback to a proposed production workflow. The assignment deliverables are mapped below.

Required deliverable	In this submission
Crew Code	Native model-dispatch records and role prompts, deterministic gates, a recorded-output runner, and the separate six-card offline controller.
Mermaid Diagram	Editable .mmd diagrams of the working case, studio oversight and proposed card pipeline, with readable renders.
README	Named game, output, folder map, run steps, dependencies and evidence limits.
Game-ready output	An existing Group 10 combat build and its card/rule data. It is a tested prototype, not an approved Unreal import or a final game.

The evidence boundary

Four separate Group 10 model calls are retained. The Lead chat used its native Agent tool to run those workers. The supplied replay makes no new model calls. The older six-card A3 controller has no live transport, and its release gates remain open. A replay is never presented as a fresh working-model run. [S1-S4]

2. THE GAME AND THE OUTPUT

Four character roles face enemies on a combat grid. Cards provide actions and resources. Combat is the first production target; dungeon encounters and landscape exploration follow.

Loop	What the crew must support	Current boundary
[Combat]	Card piles, movement, attack, defence, meaningful card choices, bots, victory, defeat and restart.	The nine versions explore one Bauglor fight. They use disclosed prototype values and do not prove complete rulebook fidelity.
[Dungeon Crawl]	Encounters, persistent tiles and the transition into combat.	Later. The demos mostly start directly in the fight rather than proving the full encounter-reveal loop.
[Landscape]	Exploration, events and the route between places.	Later. Not built or verified by this submission.

A small, concrete production task

The prepared card crew targets six stable row IDs. A stable ID lets the game, data and tests refer to the same card without guessing from its display name. It preserves raw source text separately from parsed fields.

ID	Card	Intended use
SK1_01	Knockout!	Source-checked card row; real print comparison and approval still required.
SK1_02	Double Strike	Source-checked card row; real print comparison and approval still required.
SK1_03	Mental Push	Source-checked card row; real print comparison and approval still required.
SK1_04	Salvage Resources	Source-checked card row; real print comparison and approval still required.
SK3_01	Stand as One	Source-checked card row; real print comparison and approval still required.

Copyright2026 Piero L. S. Folly | Khi-Rho Studios | Assignment3 Agent Crew Report

ID	Card	Intended use
SK4_01	Edan's Leap	Source-checked card row; real print comparison and approval still required.

What is usable now

Group 10 supplies an existing web combat build, card data, a rule audit, checks and a crew trace. The separate six-card code supplies reproducible offline diagnostics. Its intended DT_Cards.csv is withheld when print evidence is synthetic or unresolved. That refusal is useful: the crew must not turn uncertain card text into game data. [S2-S5]

Game connection is explicit. The future consumer is the F.A.M-E.D. Unreal card table. A named contract is not proof that an Unreal import has been tested.

3. THE STUDIO THAT DID THE WORK

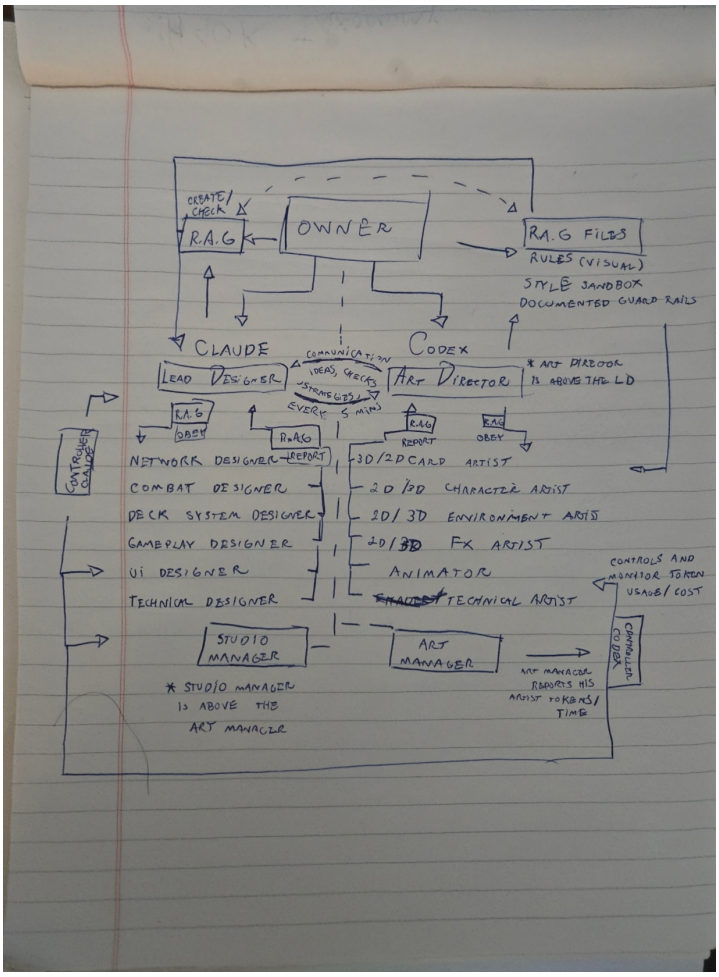
My original sketch separated design and art leadership, approved source material, specialists and cost control. The implemented studio added named inboxes, dated evidence and bounded scripts. The source drawing and the task pipeline are shown separately below.

Role	Actual responsibility	Boundary
Piero (me), studio owner	Sets the game, visual direction, priorities and final choices; plays and comments on the prototypes.	My comments stay distinct from agent interpretation. Only I choose clearing and deletion.
Studio Director, Codex	Coordinates the studio and answers the questions I delegate while I am away.	Delegated answers are labelled. It does not turn an unknown fact into my decision.
Art Director, Codex	Directs presentation, sources and bounded independent checks; coordinated the two Codex builds and this submission.	A source review is not human play. It preserves the original evidence and states what was checked.
Studio Manager, Claude	Tracks work, model settings, usage, inbox coverage, unresolved items and handoffs.	Alerts and reports are not proof of a hard security or spending boundary.
Lead Designer, Claude	Runs the Claude production stages, integrates results, maintains the shared pages and coordinates the single push.	Owns the published artifact. A queued message is not proof that a page changed.
Lead Engineer, Claude	Prepares the local environment and engine support; records technical limits.	Installed tools are not proof of a working Unreal game.
Lead Web Designer, Claude	Maintains navigation, publication surfaces and the readable artifact experience.	Presentation changes keep the existing page and its history.

A task needs more than a role name

Each task should name its game and folder, approved inputs, output format, test, budget and stop condition. The producer owns its files. A checker reports a fault before the producer corrects it. The manager tracks the handoff, and I settle a genuine design choice.

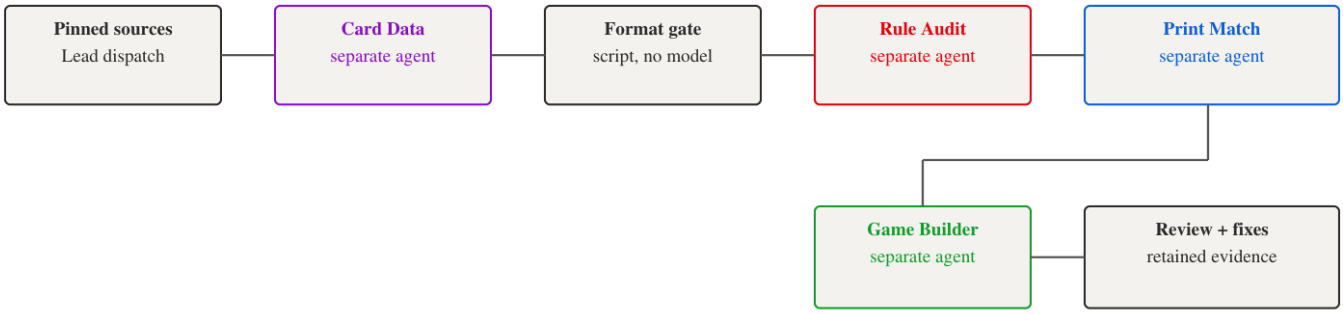
The studio worked through real model calls and human-mediated coordination. It is not a fully autonomous company. Native chat tools, local scripts and my decisions each did a different part of the work. [S3, S6]



My original design
My pen-and-paper sketch from29October. The drawing places me above Claude's Lead Designer and Codex's Art Director, with scoped source material, specialist roles and cost monitors.
Evolution into task crews
The sketch describes studio oversight. The production graphs describe a smaller task: sources become checked card data, then implementation. Controllers and dated handoffs support that flow. A drawing of RAG or a sandbox is not proof that either service was deployed.
Original image preserved in the package. The diagram below explains the selected task flow rather than replacing my sketch.

4. ONE WORKING CREW: FORMAT FIRST

Group 10 is the clearest small case for this submission. The Lead called separate workers for Card Data, Rule Audit, Print Match and Game Builder. Their original call IDs, models, task inputs and times are retained in the evidence folder. Machine-specific roots are replaced with named placeholders; the full private transcript is not included.



Working case: Group 10, native Agent calls. This is not the held six-card replay.

Initial print inputs and later corrections are preserved; no ideal blind-process claim.

Step	Input to output	Why it exists
Card Data	Approved packet -> combat-data.json	Make structured rows before writing the game.
Format gate	Rows -> pass or typed findings	A cheap script catches a wrong shape without another model call.
Rule Audit	Rows and sources -> audit.json	A second worker checks meanings and unresolved decisions.

Step	Input to output	Why it exists
Print Match	Card images and packet -> print evidence	Compare the rendered card with the data. The first Group 10 pass used art-only images; later correction evidence must stay visible.
Game Builder	Accepted data, audit, print notes and rulings -> playable build	Connect the material to an actual game loop.
Review and correction	Produced files -> defects, fixes and retained checks	A successful first build is not the end of validation.

Recorded sequence is not perfect chart compliance. Group 10 needed corrections, including last-card falling, a buff expiry case and defence use. Its initial print step did not establish the ideal blind source comparison. Those are lessons, not details to erase. [S3-S5]

5. THE CODE AND THE NEXT CARD CREW

The actual game crews used the platform Agent tool as their model transport. The included task inputs show that dispatch. Their gates and game code are ordinary source files. There was no standalone script that secretly called the model API.

The separate six-card controller is ordinary Python. It prepares a pinned packet, validates each response and keeps the first blind print read. The live transport is deliberately absent. The submission runner can reproduce recorded handoffs without spending tokens; that is labelled replay.

Mechanism	Implemented behaviour	Limit
Pinned sources	Checks the allowed inventory and file fingerprints before stages and attempts.	A fingerprint proves unchanged bytes, not that the source rule is correct.
Three distinct tasks	Card Data produces; Rule Audit checks; Print Match reads only names, IDs and pinned images in the proposed blind flow.	The older offline code alone does not prove live calls. The separately recorded fresh native run is in section13.
Response shape	Each role returns a defined JSON structure. Wrong format retries only that stage.	Four attempts per stage; the global ten-call cap wins if limits conflict.
Global caps	MAX_CALLS = 10; MAX_TOKENS = 400000 for recorded usage in replay.	No claim that this controls every studio chat or the provider account.
Stop routes	Source mismatch, usage-limit stop or exhausted retry leaves diagnostics.	No paid API, login, account switch or automatic reset fallback.
Print and release	Synthetic or unresolved print keeps QA diagnostics and suppresses CSV release.	Human review and real source/print approval remain required.

Plain orchestration recipe

```
pin_and_check_sources()
rows = call_role("Card Data", source_packet)
check_format(rows)
audit = call_role("Rule Audit", source_packet, rows)
print_read = call_role("Print Match", images_ids_names_only)
keep_first_read_unchanged(print_read)
result = validate(rows, audit, print_read)
export_only_if_eligible(result)
record_inputs_outputs_usage_and_stop_reason()
```

The recipe describes control flow. Actual dispatch records and the README identify which host tool ran the models. [S2,S3]

6. OUTPUTS, CHECKS AND FAILURE

Done means an output and evidence for that output. A model saying “pass” is one claim. A script checking the file, another worker finding a defect, and me playing the game are separate observations.

Evidence	What it supports	What it does not support
Four original native calls	Separate Group 10 role dispatches with recorded task inputs and models.	Perfect isolation, every chart box, or all rulebook rules.
Produced game and data files	The named workers produced material connected to a playable F.A.M-E.D. prototype.	Final game balance or production readiness.
Group 10 focused corrections	Retained fault reports and later checks cover specific falling, expiry and defence cases.	A complete mathematical proof of the game.
My feedback	I played the nine versions and wrote down what I liked, what confused me and what broke.	A controlled usability study or a human loss/restart test for every game.
Six-card historical tests	The retained correction receipt records 34 passes, zero failures and zero new model calls. A later preserved combined receipt records 65 passes and one skip.	A fresh test run today or a live three-model crew.

Evidence	What it supports	What it does not support
Relocated replay receipt	The exact packaged controller and assets can be checked in their new submission folder.	Real print legibility, a fresh model run, or an Unreal import.

What the card pack must preserve

The row key and display name stay separate. Raw card wording stays beside parsed values. Timing, after-use pile, tags, quantity and source references are checked. Unresolved template bindings stay unresolved. Printed exceptions are not replaced by a model's preferred rule.

A safe stop is an output

If a card is unreadable, a higher-ranked source is missing or an approval is open, the crew records the gap and stops release. It should preserve the attempted response and the reason. It should not silently create another account, call a paid service or rewrite a previous result.

The code constrains source packets. It is **not hostile-host isolation**. The person running the machine still controls the files and tools. [S2-S5]

7. NINE PROTOTYPES, DIFFERENT VARIABLES

Nine versions explored the same general Bauglor fight: two Codex builds and seven Claude builds. The planned four-plus-four comparison was not completed. Group09 and the final ninth are different builds.

Game	Builder	Engine	Nominal crew method	Main observation
02	Claude	Web	Sequential baseline	Readable log and tabs; autoplay complaint.
03	Codex	Canvas web	Adapted Chart03	Useful trace; weak visual guidance.
04	Claude	Godot	Joined data and code	Fewer redundant actions; space-bar complaint.
05	Claude	Phaser web	Separate integration lane	Strong styling; too many prompts.
06	Codex	Canvas web	Adapted Chart06	Least liked; avoid the direction.
07	Claude	Phaser web	Contract and oracle	Smooth board/card flow; adjacency confusion.
09	Claude	Three.js web	Evidence chain	Minimal 3D and camera control.
10	Claude	Phaser web	Format first	Animation and concise feedback.
ninth	Claude	Web	Sealed-oracle chain with lanes	Helpful controls; divided attention.

What changed between crews

Engine, worker count, task separation, checks, retry routes, art and interaction all varied. Some builders performed several nominal chart roles. Some independent stages ran later. The source files distinguish the planned graph from the actual trace.

Comparison limits

The set reveals interface and workflow ideas. Engines, task boundaries, checks and presentation varied together, so these results cannot establish a causal model or crew winner.

The next comparison should use the selected features and the same named acceptance cases. That future controlled test has not been run. [S4-S7]

8. OWNER FEEDBACK AND INTERPRETED GRADES

I gave written feedback, not numerical grades. The scores below are Studio Director interpretations. They are not averaged into a winner. Exact original message times are unknown.

Game	Director interpretation	My words, excerpt	Keep or address
02	8/10	the Combat log font format + colors make the game very easy to follow.	Readable log and tabs; autoplay complaint.
03	4/10	This is the weakest game that I played so far.	Useful trace; weak visual guidance.
04	8.5/10 provisional	My two most favorite so far were the Godot, the Godot one, and right now this one, the Group 10	Fewer redundant actions; space-bar complaint.
05	7.5/10	the strongest thing here is the aesthetics and the amount of icons and explanation, but that can be overwhelming too.	Strong styling; too many prompts.
06	3/10	I think this one's the one that I like the least, the number six, group six.	Least liked; avoid the direction.
07	9/10	This one has actually been the highest grade so far.	Smooth board/card flow; adjacency confusion.

Game	Director interpretation	My words, excerpt	Keep or address
09	7/10	I feel like the most outstanding thing is that you can rotate the camera on this other one in the localhost	Minimal 3D and camera control.
10	9/10	But this game has the best aesthetics in terms of interaction, animation, how being concise and in a unique language.	Animation and concise feedback.
ninth	6/10	It definitely looks like a game made for an engineer, not for a gamer.	Helpful controls; divided attention.

Preferred three prototypes

My preferred three were04Godot,07smooth2.5D/isometric and10pixel art. Preferred features were fewer redundant actions in04, board/card flow in07, and animation with concise guidance in10. Group02 adds the preferred log formatting.

Problems stay beside praise

Faults I reported include autoplay not turning off in02, a space-bar reset problem in04, missing log scroll in10, unclear diagonal adjacency in07, and divided attention in the final ninth. These faults were not newly reproduced for this document.

The supplied page data and Director record match for all nine entries. The Lead reports publication under “Piero’s grades” on Crew Tests. Art checked source and intended writes; signed-in visual display is still unverified at the source-check cutoff. [S6, S8]

9. SELECTED DESIGN AND CREW

The next design should keep the board and current action visually dominant. Extra information should explain the current choice without competing for attention.

Borrow	Use it for	Acceptance question
07: board focus and card hover	Keep the player’s eye near the action and explain a card before committing it.	Can a new player identify the active hero, legal target and card result?
10: animation and event guidance	Show what changed through concise motion, dice feedback and central prompts.	Does feedback explain the action without hiding the next choice?
04: fewer clicks	Automate a second step only when no meaningful intervening action is available.	Does automation stop when a player can legally respond?
02: readable log	Use tabs, clear type, colour and a scrollable history.	Can the player find who acted, what happened and why?
09: camera and minimal UI	Keep optional camera control and show only current information.	Can the player return to a useful view without losing state?
03, 05 and ninth: inspection tools	Keep trace, rules, seed, card details and accessibility behind clear controls.	Can the player inspect detail without moving the board out of focus?

Crew choice

The proposed crew combines source extraction, an early format gate, independent rule and print checks, a checked contract with expected cases, implementation, and independent review. Rules and presentation remain separate.

Combat before expansion

First, make one readable **[Combat]** loop with the right card decisions and reliable endings. Next, connect **[Dungeon Crawl]** encounters and persistent tiles. Only then grow the **[Landscape]**. Shared IDs and approved event contracts can carry the data forward; each phase still needs its own sources and tests.

This is a proposed design direction, **not a new implemented tenth game** or my final adoption of a crew. It follows the feedback without claiming a controlled crew winner. [S1, S6]

Why these three, and why this graph?

Preferred prototype	Workflow evidence retained	Reason for the combination
04 - Godot; Joined data and code	Checked rows and expected cases feed the builder.	Connect rule data to implementation; retain the fewer-clicks direction.
07 - smooth isometric; Contract and oracle	An allowed-value contract and hand-computed cases constrain implementation.	Make accepted rules and expected results explicit; retain board/card-hover clarity.
10 - pixel art; Format first	A deterministic format gate precedes later rule, print and build work. Independent review found defects after initial acceptance.	Reject structural errors early, then require a separate critic and recheck; retain animation and focused event guidance.

My preference selects the interface direction. Traces select useful process mechanisms. The recommended combination is therefore a proposed synthesis, not proof that one chart caused a better game. The Group10 graph in section4 documents the

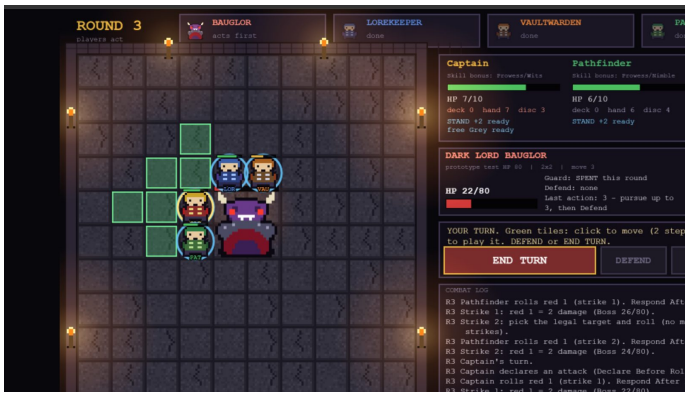
clearest retained four-worker case; it is evidence of execution, not a claim that it won every comparison.

Play-test snippets

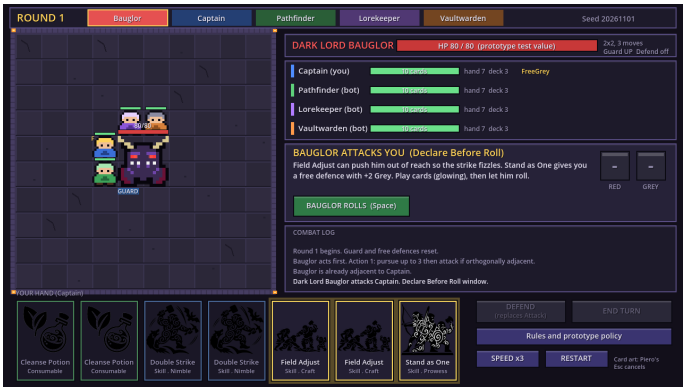
Small crops from the my play test, with the Windows watermark and browser bars removed. Each shows the feature that led to keeping that crew's mechanism.



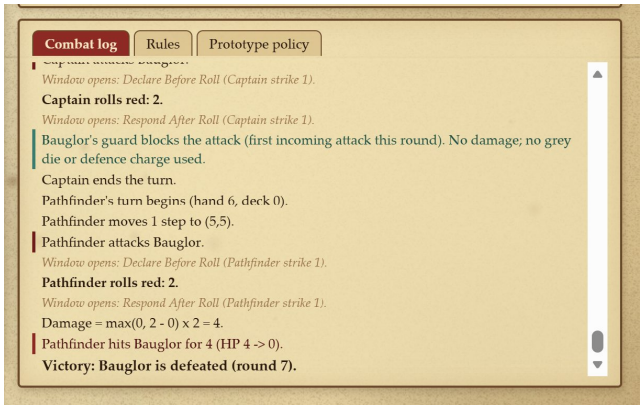
07, Contract and oracle: smooth 2.5D board and card play. Rules were fixed first, so the build stage went to presentation.



10, Format first: pixel art, guided turn and readable panels. Clean data at the start left time for interaction.



04, Joined data and code (Godot): one combat agent kept the turn flow consistent. Builder screenshot; I took none of this one.



02, Sequential baseline: the tabbed, colour-coded combat log I want reused.

Proposed combined flow

Pinned sources -> Card Data -> format gate -> independent Rule Audit and blind Print Match -> checked contract and expected cases -> builder -> independent rules/interaction review -> bounded correction and recheck. Unresolved rule or print evidence stops release. Visual changes must not alter combat rules. The editable diagram is diagrams/selected-hybrid.mmd. [S4-S7]

10. THE ARTIFACT, CUP AND CLEAR

The artifact is the readable front door. The repository and dated records keep the supporting evidence. A useful page says what changed, when it was captured, who did it, what was checked and what remains open.

Surface	Purpose	Honest limit
Studio Home	Navigation, current work, decisions and links.	A page label can become stale; it is not a substitute for the source receipt.
Crew Tests	The nine games, diagrams, reports, my feedback and comparisons.	Planned roles, actual roles, test volume and my preference must remain separate.
Game Dev Logs	Per-game changes, bugs, authors and evidence.	Later backfills keep their real logging date; they do not invent an old event time.
General Dev Log	Cross-studio events and task summaries with dates and times.	Local append, uploaded store record and visible browser row are three different states.
Mailboxes	A specific role receives a task, question, blocker or result.	A file queued is not proof that the recipient processed it. Exact handled versions suppress duplicates.

CUP: Commit, Update, Push

CUP means Commit, Update, Push: inspect and commit attributable changes, update the relevant log and handoff, then use the agreed push coordinator. Art does not push. Private runtime and unrelated files stay out.

Clear is a handoff, not deletion

A clear requires a handoff of task state, open questions, source paths, costs, mistakes and running processes. I choose the clear. The record does not establish a complete count of actual clears.

The studio caught stale counts, incorrect authorship labels, unreliable times and missed inbox messages. Additive corrections preserved the earlier record. The lesson is to report the evidence state, not just “synced”. [S7-S10]

11. KEEPING THE WORK UNDER BUDGET

The manager tracks usage and routes work. Deterministic checks do the work that does not need a model. Short role packets, bounded retries and preserved responses reduce unnecessary repeat calls. A budget alert must be actionable, not an acknowledgment loop.

Control	What it does	What I do not claim
Card controller caps	Stops its recorded-run loop at ten calls or 400000 recorded tokens; retries only format failures.	A studio-wide provider spending limit.
Codex budget reader	Reads quota metadata and produces deduplicated warnings, including an unavailable-source warning.	It does not kill model work or impose a provider-wide spending limit.
Claude budget guard	Uses saved meter readings and estimates to emit alerts.	A legacy threshold in code is not proof of my latest policy; the operator still has to stop work.
Studio Manager controller	Checks stale work, inbox coverage, watcher state, budgets and rule drift.	A security sandbox or proof all work is healthy.
Recorded task splits	Keep main coordination separate from direct game workers.	A complete price per game or a fair quality-per-token score.
Reserve and hold	Preserve at least 10% of Codex weekly allowance; no automatic reset, account switch or paid fallback.	That an account-wide meter change is this task’s cost.

Code: tools/claude-budget/budget_guard.py; tools/codex-inbox-watch/budget.mjs; tools/studio-manager/controller.py; and the card controller run_crew.py. The first three observe and report. The card controller can stop its own run. Watchdog remains OFF. Switch-sync live writes remain under my pause; an individual switch file can still say enabled.

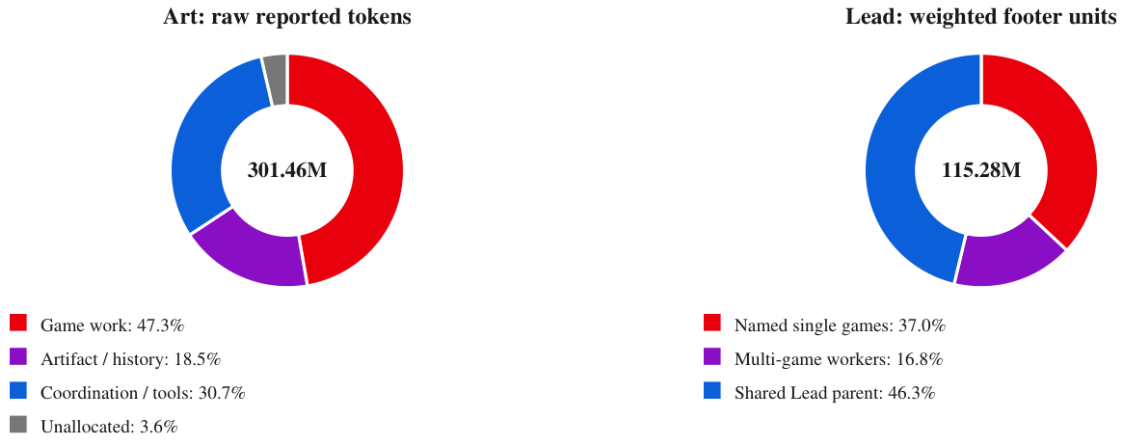


Chart: **recorded workload, not elapsed time**. The left groups Art’s task-window records into game work, artifact/history work, coordination/infrastructure and unallocated work. It includes repeated cached input and is a correlation, not a bill. The right uses Lead’s weighted footer units from its separate snapshot. Each chart has its own total and scale. They are never added; shared work is not split equally between games.

No reliable active-time split or per-task dollar bill is available. The charts retain measured workload units only. Lead reports publication under Crew Tests -> Workload by task; Art checked the supplied page copy and asset fingerprint. Independent viewer access remains unverified. [S7,S11]

12. THE JOURNEY AND WHAT CHANGED

Period	Work and change	Lesson
26-28 October	Source rules, GDD revisions, six-card scope, response schemas and source boundaries were prepared.	A stable row ID and a pinned source packet are better than a large unfiltered folder.
28-29 October	The offline controller, retry routes, print boundary and failure cases were corrected.	A synthetic pass must not become a real print or live-run claim.
31 October	Document/code review, activity capture, handoffs and watchdog work exposed coordination gaps.	A role needs a specific inbox and an honest result, not a vague message to “Claude”.
1 November, overnight	Crews produced nine combat variants with different charts, engines and interfaces.	A method on paper is not the same as its executed graph. Unequal crews cannot prove a model winner.
1 November, my play test	My reports shifted the emphasis toward board focus, readable feedback and fewer clicks.	A game that passes scripts can still be confusing or unpleasant to play.
1 November, close-out	Logs, attribution, counts, commits, token splits and mistakes were preserved before clearing.	Preserve the reason and evidence for a correction, not just the final green label.

Mistakes that improved the system

A sender-prefix filter missed legitimate messages. A blurred screenshot produced an unreliable log count. Some later records attributed work to the wrong actor. A source-only check was at risk of being reported as live confirmation. Each mistake led to a narrower evidence label or a better check.

The merged mistakes register has 77 recorded rows across six chats at its saved cutoff. Rows include incidents, blocks and game defects. They can overlap across chats, and environment blocks are not model mistakes. The total is **not an AI error rate**.

Time spent

The record spans several calendar days, not continuous labour. Group10’s saved stage durations add to about33minutes, while its event span is7hours20minutes43seconds including idle gaps. Neither establishes whole-studio labour or an artifact-versus-game time split.

13. A FRESH THREE-AGENT CARD RUN

The fresh run I authorized used three distinct native Codex workers: Card Data, Rule Audit and blind Print Match. Each returned six structured rows. There was one call per role, no retry, no paid API and no new game.

Role	Actual input and result	Evidence boundary
Card Data, Sol Low	Read one pinned source packet; returned six exact-text card rows plus fields and unresolved items.	Correct response shape did not mean valid game data. The deterministic gate rejected provenance spellings and unsupported effect tags.
Rule Audit, Sol High	Read independent pinned sources and the fresh producer rows. Flagged four source reviews; passed two rows.	Flags are review findings, not four automatically confirmed bugs. The missing structured value on Knockout needs interpretation rather than an invented ruling.
Print Match, Sol High	Viewed only six exact pinned image crops, IDs and names. Read five; marked Mental Push unreadable.	No producer or audit text was supplied. This proves screen-image delivery; it does not prove physical printed-card approval.

The result: stop before release

STOPPED_FOR_OWNER; no CSV emitted. The existing deterministic validator checked all three outputs without a crash. It retained the first blind response unchanged. Source review flags, invalid producer tags/provenance and unreadable print blocked release. No source was changed to make a green result. Group10 remains the supplied playable game-output case.

What the critic caught

Double Strike says +4 in exploration text but its structured value says3. Salvage Resources names the R.F.D. pile in text while a structured field names discard. Mental Push has unresolved reward symbols and a round/turn disagreement. These are specific, cited findings for later source resolution. The crew kept raw text rather than guessing a fix.

Actual controls and limits

The native workers had instructed read-only access to one packet or the six images. This is an adaptation of the older tool-less plan, not a proven hostile-host sandbox. Per-worker tokens were unavailable and remain null. The integer zero used only inside the old validator’s schema adapter is not measured usage. Art capped dispatches at three, used no retries and checked the account meter at84% used,16% left. The old400000-token cap was not enforced on unknown native usage.

Inspect **live-run/rows.json**, **audit.json**, **print-match.json**, **dispatch.json**, **validate_live.py** and **checked/receipt.json**. The new native recipe documents the host boundary; it does not claim a standalone live API transport. Existing synthetic fixtures and the original offline controller are preserved. This one run closes the “never ran three models” gap, not release approval or complete assignment credit. [S12]

14. README AND HOW TO INSPECT THE CREW

Folder or file	What to inspect
README.md	Start here: game, output, required software, replay command and limitations.
crew-code/	Readable native role dispatch recipe, captured role prompts, deterministic checks and recorded-output runner. No secret model API transport.
evidence/	Selected dispatch records, source fingerprints, run receipts and case-study references. No full private transcript or hidden answer key.
diagrams/	Editable Mermaid sources and readable diagram images.
game-output/	The existing selected combat build and the data produced by its crew. It is not rebuilt for this report.
six-card-offline/	Explicitly allowlisted prepared controller, sources, schemas, roles and synthetic fixture. Live transport remains blocked.
submission-manifest.json	Exact included files and their fingerprints, plus excluded sensitive/review-only categories.

Inspection sequence

1. Read the evidence labels and the source register. 2. Inspect the four distinct native Agent calls and their produced files. 3. Read the role prompts and diagrams. 4. Run the supplied recorded-output check in a fresh output folder. 5. Inspect its report. The README gives the exact tested commands and exit states.

What replay means

Replay loads retained answers rather than asking a model again. It makes the sequence reproducible without spending tokens. It can prove that the packaged code handles those records without crashing. It cannot prove fresh model behaviour, readable source pixels, balanced human play or a working engine import.

What remains open

The held six-card run was released and completed through native Codex workers; see section13. Standalone live API transport, real print approval, Unreal import and broader game fidelity remain open. The page may show newer information than this frozen report. Exact outcomes are recorded in the submission receipt, not invented in this prose.

This is a game-connected, evidence-led submission. It shows real studio work and usable outputs while leaving the remaining engineering and rubric risks visible.

15. SOURCES AND FINAL READING GUIDE

This register points to the evidence used for the report. The submission manifest supplies exact included file fingerprints. Repository paths refer to the source project; the README maps the exported copies. These are sources, not new tests.

ID	Source	Used for
S1	GDD Draft 2 Rev5 PDF; gdd_draft2_rev5.py	Game connection, phase colours, Georgia type, tables and layout. No source GDD was changed.
S2	assignments/a3-agent-crew: README, run_crew.py, packet_support.py, source_policy.py, validate.py, contract and retained correction/review receipts	Prepared three-role controller, safe replay, source boundary, output contract and open live-run gates.
S3	Selected original Lead Agent calls; group10-native-agent-dispatch.json	Four actual role dispatches. Roots are substituted; full private transcripts stay local.
S4	crew-tests/group-10: crew-trace.json, stages, GAME-SUMMARY.json, code and report	Actual production case, files, script gates, defects and corrections.
S5	crew-tests/*: GAME-SUMMARY.json, crew-trace.json and build reports	Nine variants, actual authorship and qualified implementation evidence.
S6	owner-game-verdicts-director-2026-11-01-v1.json	My written words, separate Director scores and proposed design synthesis.
S7	crew-tests/PLAN.md; BUDGET-TABLE.md; named role handoffs	Original goals, changed scope, budget records and operational limits.
S8	Crew Tests page22source, grades write-copy and grades-section-v22-art-check.json	Nine-row source comparison; publication/read-back remains attributed to Lead.
S9	Studio Dev Log entries and append-only events; count-investigation and CUP receipts	Dated journey, additive corrections and separate publication states.

ID	Source	Used for
S10	tools/studio-manager; studio-devlog; codex-inbox-watch; watchdog and switch-sync records	What local controllers and monitors do, including their off/paused states.
S11	chat-costs-by-game-art-director-2026-11-01.json; chat-costs-by-game-lead-2026-11-01.json; mistakes-merged-2026-11-01.json	Separated workload units, cutoff limits and the preserved incident register.

Instructor materials are included in the ZIP; no access to my Claude account is needed. Native viewer-share links remain unverified and are omitted.

S12: live-run/dispatch.json, three retained actual responses and checked/receipt.json. The run I released was validated November1 at17:46 PST; no synthetic response substitution or CSV release.